

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное образовательное учреждение высшего образования
САНКТ – ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

КАФЕДРА № 42

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

Старший преподаватель

должность, уч. степень, звание

подпись, дата

С.Ю. Гуков

инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №9

Асимметричный алгоритм шифрования RSA

по курсу: Алгоритмы и структуры данных

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. №

4321

подпись, дата

Г.В. Буренков

инициалы, фамилия

Санкт-Петербург 2024

1 Цель работы

Цель работы заключается в изучении принципов асимметричного шифрования и реализации программы, которая выполняет шифрование и расшифровку текстовых сообщений с использованием алгоритма RSA. В рамках выполнения работы необходимо продемонстрировать процесс генерации ключей, зашифровать текстовые данные с помощью открытого ключа и расшифровать их с использованием закрытого ключа, а также проверить корректность алгоритма на примерах.

2 Задание

Задание заключается в реализации программы, использующей асимметричный алгоритм шифрования RSA. Программа должна генерировать пару ключей (открытый и закрытый), выполнять шифрование текстовых сообщений с использованием открытого ключа и расшифровку зашифрованного текста с применением закрытого ключа. Также необходимо продемонстрировать корректную работу алгоритма на примере текстового сообщения, обеспечив соблюдение всех условий безопасности, таких как выбор подходящих простых чисел и проверка корректности ключей.

.

3 Краткое описание хода работы

В ходе выполнения работы была разработана программа на языке JavaScript, реализующая асимметричный алгоритм шифрования RSA. Основные этапы включали генерацию пары ключей (открытого и закрытого), выполнение шифрования текстового сообщения и последующую расшифровку данных. Для генерации ключей использовались случайные простые числа, при этом проверялось выполнение необходимых условий, таких как взаимная простота выбранных чисел.

Шифрование текста осуществлялось путем преобразования символов в их ASCII-коды и применения формулы шифрования с использованием открытого ключа. Для расшифровки применялся обратный процесс, использующий закрытый ключ, что позволяло восстановить исходное сообщение. Код был структурирован в виде функций для каждого этапа алгоритма, включая генерацию ключей, шифрование, расшифровку и проверку входных данных.

Для проверки работоспособности программы были выполнены тесты на примерах текстовых сообщений. Программа корректно обрабатывает текстовые данные, обеспечивая безопасность алгоритма и точность восстановления исходного сообщения после расшифровки.

4 Исходный код

Ниже предоставлен исходный код моей программы с комментариями основных блоков:

```
const readline = require('readline');
```

```
const rl = readline.createInterface({  
  input: process.stdin,  
  output: process.stdout,  
});
```

```
function isPrime(num) {  
  if (num < 2) return false;  
  for (let i = 2; i <= Math.sqrt(num); i++) {  
    if (num % i === 0) return false;  
  }  
  return true;  
}
```

```
function generateRandomPrime(min = 3, max = 50) {  
  let prime;  
  do {  
    prime = Math.floor(Math.random() * (max - min + 1)) + min;  
  } while (!isPrime(prime));  
  return prime;  
}
```

```
function gcd(a, b) {  
  return b === 0 ? a : gcd(b, a % b);  
}
```

```

function modInverse(e, phi) {
    let m0 = phi, t, q;
    let x0 = 0, x1 = 1;

    while (e > 1) {
        q = Math.floor(e / phi);
        t = phi;
        phi = e % phi;
        e = t;
        t = x0;
        x0 = x1 - q * x0;
        x1 = t;
    }

    return x1 < 0 ? x1 + m0 : x1;
}

function modExp(base, exp, mod) {
    let result = 1;
    base = base % mod;
    while (exp > 0) {
        if (exp % 2 === 1) {
            result = (result * base) % mod;
        }
        exp = Math.floor(exp / 2);
        base = (base * base) % mod;
    }
    return result;
}

```

// Функция для проверки максимального ASCII-кода в сообщении

```
function getMaxAsciiCode(message) {  
    return Math.max(...message.split("").map(char => char.charCodeAt(0)));  
}
```

// Генерация ключей с автоматическим подбором p и q

```
function generateKeys(message) {  
    const maxAsciiCode = getMaxAsciiCode(message);  
  
    let p, q, n;  
    do {  
        p = generateRandomPrime();  
        q = generateRandomPrime();  
        n = p * q;  
    } while (n <= maxAsciiCode); // Проверка, что n больше максимального  
ASCII-кода
```

```
    const phi = (p - 1) * (q - 1);
```

```
    let e;
```

```
    do {  
        e = Math.floor(Math.random() * (phi - 2)) + 2;  
    } while (gcd(e, phi) !== 1);
```

```
    const d = modInverse(e, phi);
```

```
    return { p, q, n, e, d };  
}
```

```
function encrypt(message, publicKey) {
```

```

const { e, n } = publicKey;

// Проверка, что все символы в допустимом диапазоне
for (const char of message) {
  if (char.charCodeAt(0) >= n) {
    throw new Error(
      `Ошибка: Символ "${char}" с ASCII-кодом ${char.charCodeAt(0)}
превышает модуль n = ${n}. Выберите другие p и q.`
    );
  }
}

return message
  .split("")
  .map(char => modExp(char.charCodeAt(0), e, n));
}

function decrypt(cipher, privateKey) {
  const { d, n } = privateKey;
  return cipher
    .map(num => String.fromCharCode(modExp(num, d, n)))
    .join("");
}

function main() {
  rl.question('Введите сообщение для шифрования: ', message => {
    if (!message) {
      console.log("Сообщение не может быть пустым. Попробуйте снова.");
      return rl.close();
    }
  });
}

```



```

const keys = generateKeys(message);

console.log(`Программа выбрала простые числа: p = ${keys.p}, q =
${keys.q}`);

console.log(`Открытый ключ (e, n): (${keys.e}, ${keys.n})`);
console.log(`Закрытый ключ (d, n): (${keys.d}, ${keys.n})`);

try {
    const publicKey = { e: keys.e, n: keys.n };
    const privateKey = { d: keys.d, n: keys.n };

    const encrypted = encrypt(message, publicKey);
    console.log(`Зашифрованное сообщение: ${encrypted}`);

    const decrypted = decrypt(encrypted, privateKey);
    console.log(`Расшифрованное сообщение: ${decrypted}`);

    if (decrypted !== message) {
        console.error("Ошибка: расшифрованное сообщение не совпадает с
исходным. Попробуйте заново.");
    }
} catch (error) {
    console.error(error.message);
    console.log("Попробуйте заново с другими параметрами.");
}

rl.close();
});
}

```

```
main();
```

5 Результат работы с примерами

В следующих скриншотах будут предоставлены результаты тестирования программы

```
Введите сообщение для шифрования: Йоуууу чувака, this game f01041      2391dka !!##
Программа выбрала простые числа: p = 41, q = 37
Открытый ключ (e, n): (299, 1517)
Закрытый ключ (d, n): (419, 1517)
Зашифрованное сообщение: 644,1310,720,720,720,720,1034,721,720,815,813,1043,813,1011
34,497,497,1319,1319
Расшифрованное сообщение: Йоуууу чувака, this game f01041      2391dka !!##
```

Рисунок 1 – пример один

```
Введите сообщение для шифрования: 1      0      2 # как дела новый каделак
Программа выбрала простые числа: p = 31, q = 47
Открытый ключ (e, n): (587, 1457)
Закрытый ключ (d, n): (623, 1457)
Зашифрованное сообщение: 665,50,424,50,404,764,404,822,404,753,138,753,404,911,1455,430,138,404,1271,311,1299,444,1410,404,753,138,911,1455,430,138,753
Расшифрованное сообщение: 1      0      2 # как дела новый каделак
```

Рисунок 2 – пример два

6 Выводы

В результате выполнения лабораторной работы была разработана программа, реализующая алгоритм шифрования и расшифровки RSA. Программа успешно генерирует ключи, корректно шифрует текстовые сообщения с использованием открытого ключа и выполняет их расшифровку с применением закрытого ключа.

Тестирование программы на различных примерах показало её работоспособность и соответствие требованиям алгоритма RSA. Реализация продемонстрировала надежность обработки данных и правильность восстановления исходного сообщения, а также позволила изучить основные принципы асимметричного шифрования.

Полученные результаты подтверждают эффективность алгоритма и его пригодность для защиты информации, а также укрепляют навыки работы с криптографическими методами и их программной реализацией.